

Human Computer Interaction



Table of Contents

<i>Lesson 1: Introduction to HCI & Interaction Design</i>	9
1. What is Human–Computer Interaction (HCI)?	9
2. Goals of HCI.....	9
3. Core Principles of HCI.....	9
4. User-Centered Design (UCD)	10
5. Key Concepts in HCI.....	10
6. Common Input Devices in HCI	11
7. Graphical User Interface (GUI).....	11
8. Example of Direct Manipulation.....	11
9. Why HCI is Important	11
Quick Exam Summary (Important)	12
Possible Exam Questions	12
<i>Lesson 2: Gathering Data & User Analysis</i>	12
1. Why Gather Data in HCI?	12
2. Methods for Gathering User Data.....	13
3. Persona	14
4. User Story	14
5. Scenario	15
6. Storyboard	15
7. Why Personas, Stories and Scenarios are Important	16
Quick Exam Summary	16
Possible Exam Questions	16
<i>Lesson 3: Interfaces in Human-Computer Interaction</i>	17
1. What is an Interface?	17
2. Types of User Interfaces	17
3. Direct Manipulation.....	19
4. Multimodal Interfaces.....	19
5. Challenges in Interface Design	20

6. Good Interface Design Principles.....	20
Quick Exam Summary	21
Possible Exam Questions	21
<i>Lesson 4: Prototyping in HCI</i>	<i>21</i>
1. What is a Prototype?	21
2. Why Prototyping is Important	22
3. Types of Prototypes.....	22
4. Storyboarding (Linked to Prototyping)	23
5. Example (From Your Coursework)	23
6. Wizard of Oz Prototyping.....	24
7. Benefits of Prototyping.....	24
Quick Exam Summary	24
Possible Exam Questions	24
<i>Lesson 5: Usability & Evaluation</i>	<i>25</i>
1. What is Evaluation?	25
2. Why Evaluation is Important.....	25
3. Types of Evaluation	25
4. Formative vs Summative Evaluation.....	26
5. Usability Testing	27
6. Important Evaluation Terms.....	27
7. Nielsen's Important Heuristics	28
8. Ethics in Evaluation	28
9. Other Important Exam Points.....	29
10. Benefits of Evaluation.....	29
<i>Lesson 6: UX in the Software Life Cycle / Agile UX.....</i>	<i>30</i>
1. What is UX in the Software Life Cycle?.....	30
2. What is Agile Development?	31
3. What is Agile UX?	31
4. Key Ideas of Agile UX	31
5. User Research in Agile UX.....	32

6. Iteration 0.....	32
7. Parallel Tracks Approach	32
8. Advantages of Parallel Tracks	33
9. Documentation in Agile UX.....	33
10. Technical Debt in UX.....	33
11. Lean UX	34
12. Discovering Requirements	34
13. User Stories.....	35
14. Personas and Scenarios	35
15. Choosing Among Design Alternatives	36
16. Risks if UX is Ignored.....	36
17. UX in Testing and Maintenance	36
18. Benefits of Integrating UX into Development	37
Quick Exam Summary	37
Possible Exam Questions	37
<i>Lesson 7 — Visualisation / Good UX through Visual Design</i>	<i>38</i>
1. What is Visualisation?	38
2. Why Visualisation is Important	38
3. Information Visualisation (InfoVis).....	38
4. What Good Visualisation Should Do	39
5. Examples from the Lecture.....	39
6. Visualisation in Interface Design	40
7. Text List vs Visual Grid	40
8. Product Information Display	41
9. Decision Support Through Visualisation.....	41
10. Visual Feedback in Interfaces.....	41
11. Why Visual Cues Matter	42
12. Visualisation and UX.....	42
13. Important Exam Ideas.....	42
Quick Exam Summary	43

Possible Exam Questions	43
<i>Lesson 8 — Computer Graphics: Line and Circle Drawing Algorithms</i>	43
1. Why Line and Circle Drawing Matter	43
2. What is the Line Drawing Problem?	44
3. DDA Algorithm	44
4. Slope in DDA	45
5. Bresenham's Line Algorithm.....	45
6. DDA vs Bresenham	46
7. Circle Drawing Algorithms	47
8. Midpoint Circle Algorithm	47
9. Output of Midpoint Circle Algorithm.....	48
10. Anti-Aliasing	48
11. HCI Connection.....	49
12. Must-Remember Exam Points	49
Quick Exam Summary	49
Possible Exam Questions	50
<i>Lesson 9 — Digital Image Processing.....</i>	50
1. What is a Digital Image?	50
2. What is Digital Image Processing?	50
3. Why Image Processing is Important	51
4. Common Types of Digital Images	51
5. Image Enhancement	52
6. Image Compression	52
7. Segmentation	53
8. Object Recognition	54
9. Edge Detection	54
10. Region Detection / Segmentation.....	54
11. Colour Image Processing.....	55
12. Interactive Features Using Image Processing.....	55
13. Applications of Image Processing	55

14. HCI Connection.....	56
15. Must-Remember Exam Points	56
Quick Exam Summary	56
Possible Exam Questions	57
<i>Lesson 10 — Java Basics for HCI / OOP in Java.....</i>	<i>57</i>
1. What is Java?.....	57
2. What is the JVM?.....	58
3. Java is Object-Oriented.....	58
4. Class and Object	58
5. Methods and Variables	59
6. Java Syntax Basics.....	59
7. The main() Method	60
8. Constructor	60
9. Creating an Object	60
10. Encapsulation.....	61
11. Inheritance.....	61
12. Polymorphism	62
13. Abstraction	62
14. Access Modifiers	62
15. Non-Access Modifiers.....	63
16. Scanner Class	63
17. ArrayList	64
18. Import Statement.....	64
19. Why Java Matters in HCI.....	64
20. Must-Remember Exam Points	65
Quick Exam Summary	65
Possible Exam Questions	65
<i>Lesson 11 — Java Swing & Building GUIs.....</i>	<i>66</i>
1. What is Java Swing?	66
2. Why Swing is Important in HCI.....	66

3. Common Swing Components	66
4. Common Component Methods.....	68
5. Event Handling	68
6. JOptionPane	69
7. Other Useful Swing Components	69
8. Graphics in Swing	70
9. DDA and Midpoint Circle in Java.....	70
10. Transformations in Java Swing.....	70
11. Typical Structure of a Swing Program	71
12. Why Swing Matters for the Exam	71
13. Must-Remember Exam Points	71
Quick Exam Summary	72
Possible Exam Questions	72
<i>Lesson 12 — OpenGL / JOGL Basics in Java.....</i>	<i>73</i>
1. What is JOGL?	73
2. Why JOGL is Important	73
3. OpenGL Setup in Java	73
4. GLEventListener.....	74
5. Purpose of the GLEventListener Methods	74
6. GL2 Object.....	74
7. Drawing Shapes in JOGL	75
8. Drawing a Square with Lines	75
9. GLProfile, GLCapabilities, and GLCanvas.....	75
10. GLJPanel.....	76
11. Displaying JOGL in a Window	76
12. Scaling in JOGL	76
13. Rotation in JOGL	76
14. Lighting in JOGL	77
15. Animation in JOGL	77
16. Why OpenGL Matters in HCI	77

17. Difference Between Swing Graphics and JOGL.....	78
18. Must-Remember Exam Points.....	78
Quick Exam Summary	79
Possible Exam Questions	79

Lesson 1: Introduction to HCI & Interaction Design

1. What is Human–Computer Interaction (HCI)?

Human-Computer Interaction (HCI) is the field that studies **how people interact with computers and digital systems**.

The main goal is to **design systems that are easy, efficient, and enjoyable to use**.

Example:

Mobile apps, websites, ATMs, touch screens, smart devices.

The focus of HCI is **improving usability and interaction between humans and computers**.

2. Goals of HCI

HCI aims to create systems that are:

- **Easy to learn**
- **Efficient to use**
- **Safe and error-tolerant**
- **Enjoyable for users**

These goals improve **user experience (UX)**.

3. Core Principles of HCI

Important principles that guide good interface design:

Usability

How easily users can learn and use a system.

User Experience (UX)

The overall experience users have when interacting with a system.

Accessibility

Designing systems that **people with disabilities can use**.

4. User-Centered Design (UCD)

User-Centered Design means **designing systems based on user needs and preferences.**

Users are involved throughout the design process.

Steps usually include:

1. Understanding users
2. Designing solutions
3. Testing with users
4. Improving the design

5. Key Concepts in HCI

Affordance

A property of an object that **suggests how it should be used.**

Example:

A button looks clickable.

Feedback

The system informs users about the **result of their actions.**

Example:

- Loading bars
- Confirmation messages
- Error alerts

Cognitive Load

The **mental effort required to use an interface.**

Good design tries to **reduce cognitive load.**

6. Common Input Devices in HCI

Examples:

- Mouse
- Keyboard
- Touchscreen
- Voice input
- Gesture controls

The mouse is one of the common input devices studied in HCI.

7. Graphical User Interface (GUI)

GUI stands for **Graphical User Interface**.

It allows users to interact using:

- icons
- windows
- menus
- buttons

Example:

Windows operating system interface.

8. Example of Direct Manipulation

Direct manipulation means users interact **directly with objects on the screen.**

Example:

Dragging files into folders.

9. Why HCI is Important

Good HCI design:

- Improves usability
 - Reduces user errors
 - Saves time
 - Increases satisfaction
 - Makes systems easier to learn
-

Quick Exam Summary (Important)

Remember these keywords:

HCI
Usability
User Experience (UX)
User-Centered Design
Affordance
Feedback
Cognitive Load
GUI
Accessibility

Possible Exam Questions

You might get questions like:

1. What is Human-Computer Interaction?
 2. What is affordance in interface design?
 3. What is the purpose of feedback in HCI?
 4. What is user-centered design?
 5. What is cognitive load?
-

Lesson 2: Gathering Data & User Analysis

1. Why Gather Data in HCI?

Before designing a system, designers must **understand the users**.

User research helps to:

- understand user needs

- identify problems users face
- design systems that match real behavior

User research is the **first step in designing around users**.

2. Methods for Gathering User Data

Common techniques used in HCI:

Interviews

Talking directly with users to understand their experiences and needs.

Example questions:

- What problems do you face using this app?
 - What features do you want?
-

Questionnaires / Surveys

Users answer a set of questions.

Advantages:

- Collect data from many users quickly
 - Easy to analyze results
-

Observation

Watching users while they perform tasks.

This helps designers understand:

- real behavior
 - mistakes users make
 - usability issues
-

User Testing

Users interact with a system while researchers observe their behavior.

This helps identify usability problems.

3. Persona

A **persona** is a fictional character that represents a typical user of the system.

Personas help designers understand **different types of users**.

Example Persona

Name: Sarah

Age: 25

Job: University Student

Goals:

- Buy furniture online easily

Problems:

- Confusing navigation
- Too many options

Personas help designers **focus on real user needs**.

4. User Story

A **user story** is a short description of what a user wants to do.

It explains:

- who the user is
- what they want
- why they want it

Format

As a **[type of user]**

I want to **[do something]**

So that **[goal]**

Example

As a **customer**

I want to **search furniture by category**

So that **I can find items quickly**.

5. Scenario

A **scenario** describes how a user performs a task in a system.

It explains the **context and motivation of the user**.

Example Scenario

Mary wants her daughter Amy to learn about cultural heritage.

She **uses** a mobile app that allows users to **experience historical events using augmented reality**.

This scenario explains:

- the user
 - the goal
 - how the system helps
-

6. Storyboard

A **storyboard** is a visual sequence showing how users interact with a system.

It uses:

- sketches
- screenshots
- illustrations

Storyboards help designers **visualize user interactions step by step**.

Example:

1. User opens the app
2. User searches for a product
3. User selects an item

4. User completes purchase

7. Why Personas, Stories and Scenarios are Important

They help designers:

- understand user behavior
- identify user needs
- predict usability problems
- design better user experiences

Everything that helps designers **get closer to users improves design decisions.**

Quick Exam Summary

Important terms:

Persona
User Story
Scenario
Storyboard
User Research
Observation
Interviews
Questionnaires

Possible Exam Questions

You might get questions like:

1. What is a persona in HCI?
 2. What is a user story?
 3. What is the purpose of a storyboard?
 4. Why is user research important?
 5. What is a scenario in HCI design?
-

Lesson 3: Interfaces in Human-Computer Interaction

1. What is an Interface?

An **interface** is the point where a **user interacts with a computer system**.

It allows users to:

- give commands
- receive information
- control system functions

Examples:

- Mobile apps
 - Websites
 - ATM machines
 - Desktop software
-

2. Types of User Interfaces

Command Line Interface (CLI)

Users interact by **typing commands**.

Example:

```
cd documents  
mkdir folder
```

Characteristics:

- Requires technical knowledge
- Very powerful for experts
- No graphical elements

Example systems:

Linux Terminal, Windows Command Prompt.

Graphical User Interface (GUI)

A **GUI uses visual elements** like icons, buttons, menus, and windows.

Examples:

- Windows OS
- macOS
- Mobile phone interfaces

Advantages:

- Easy to learn
- Visual interaction
- Suitable for beginners

GUI stands for **Graphical User Interface**.

Menu-Based Interface

Users choose options from **menus**.

Examples:

- ATM machines
- Restaurant ordering kiosks
- Phone IVR menus

Advantages:

- Easy navigation
 - No typing required
-

Touch User Interface

Users interact through **touch gestures**.

Examples:

- Smartphones
- Tablets
- Self-service kiosks

Common gestures:

- Tap
 - Swipe
 - Pinch
 - Drag
-

Voice User Interface (VUI)

Users interact using **voice commands**.

Examples:

- Alexa
- Siri
- Google Assistant

Challenge:

Voice systems sometimes struggle with **different accents or speech patterns**.

3. Direct Manipulation

Direct manipulation means users **interact directly with objects on the screen**.

Example:

- Dragging files into folders
- Resizing images
- Moving icons

Benefits:

- Easy to understand
- Immediate feedback
- More intuitive

Example:

Dragging and dropping files on a desktop.

4. Multimodal Interfaces

Multimodal interfaces use **multiple interaction methods together**.

Example:
Voice + Touch.

Example scenario:
A user says
“Open maps” and then taps a location.

Multimodal interaction improves **accessibility and flexibility**.

5. Challenges in Interface Design

Designers must consider:

Limited Screen Space

Especially on mobile devices.

Touch Accuracy

Small buttons are hard to tap.

User Errors

Users can misunderstand controls.

Mobile interfaces often suffer from **limited screen space and touch accuracy issues**.

6. Good Interface Design Principles

A good interface should be:

- Simple
- Consistent
- Easy to learn
- Efficient to use
- Provide feedback

Consistency ensures **similar actions produce similar results across the interface**.

Quick Exam Summary

Important terms to remember:

Interface

CLI

GUI

Menu-Based Interface

Touch Interface

Voice Interface

Direct Manipulation

Multimodal Interaction

Consistency

Possible Exam Questions

1. What is a user interface?
 2. What is a GUI?
 3. What is direct manipulation?
 4. What is multimodal interaction?
 5. What are challenges in mobile interface design?
-

Lesson 4: Prototyping in HCI

1. What is a Prototype?

A **prototype** is a **simple version of a system** created to test ideas before full development.

It helps designers:

- explore ideas
- test usability
- get user feedback

The main goal is to **test and refine ideas early**.

2. Why Prototyping is Important

Prototyping helps to:

- Identify design problems early
 - Save time and cost
 - Improve usability
 - Test ideas before coding
 - Get user feedback quickly
-

3. Types of Prototypes

Low-Fidelity Prototype

Simple and rough design.

Examples:

- Paper sketches
- Hand-drawn interfaces
- Wireframes

Features:

- Quick to create
- Cheap
- Focus on layout and ideas

Used to **quickly explore design ideas.**

High-Fidelity Prototype

Detailed and realistic design.

Examples:

- Interactive apps
- Clickable UI designs
- Fully styled interfaces

Features:

- Looks like the final product
- More expensive
- Takes more time

⚠ Disadvantage:

People may think the design is **already final**.

4. Storyboarding (Linked to Prototyping)

A **storyboard** shows how users interact with a system step-by-step.

It is a **visual story of user experience**.

Used to:

- understand user journey
- identify usability problems
- communicate ideas clearly

Storyboards help visualize **user interaction scenarios**.

5. Example (From Your Coursework)

In your furniture store system:

Storyboard steps:

1. User opens the website
2. User browses furniture
3. User selects item
4. User adds to cart
5. User completes checkout

This matches your evaluation task where you must:

- create storyboards
 - design paper prototypes
 - gather feedback
-

6. Wizard of Oz Prototyping A human secretly simulates system functions.

A special type of prototype where:

A human secretly simulates system functions.

Example:

- User thinks AI is responding
- But a human is actually controlling responses

Used to test ideas **before building real systems.**

7. Benefits of Prototyping

- Early feedback from users
 - Better design decisions
 - Reduced development risk
 - Improved user satisfaction
-

Quick Exam Summary

Key terms:

Prototype
Low-fidelity
High-fidelity
Storyboard
Wizard of Oz
User testing
Feedback

Possible Exam Questions

1. What is a prototype in HCI?
2. Difference between low and high-fidelity prototypes
3. What is storyboarding?
4. What is Wizard of Oz prototyping?
5. Why is prototyping important?

Lesson 5: Usability & Evaluation

1. What is Evaluation?

Evaluation is the process of checking whether a system, prototype, or interface works well for users.

It is used to find out:

- whether users can use the product
- whether the product meets user needs
- whether it should be improved

Evaluation is used throughout design to check if users can use and like the product.

2. Why Evaluation is Important

Evaluation helps designers:

- Find usability problems
- Improve designs early
- Check whether user needs are met
- Compare different designs
- Make the final product better

3. Types of Evaluation

Usability Testing

Users perform tasks while researchers observe them.

Used for:

- measuring performance
- finding errors
- checking satisfaction

Common things measured:

- task completion time
 - error rate
 - success rate
 - user satisfaction
-

Field Study Evaluation in a natural setting

Evaluation in a **natural setting**.

Examples:

- home
- workplace
- public environment

Purpose:

- understand real behaviour
 - see how users interact naturally with technology
-

Heuristic Evaluation Experts inspect an interface using usability rules.

Experts inspect an interface using usability rules.

Important:

- no real users required
- useful for finding design problems quickly

Heuristic evaluation identifies usability problems using a set of guidelines.

4. Formative vs Summative Evaluation

Formative Evaluation

Done **during design/development**.

Purpose:

- improve the product while it is being made
-

Summative Evaluation

Done **after the product is completed.**

Purpose:

- judge the final quality or success of the product

This distinction appears in your module materials and is important for exams.

5. Usability Testing A usability test involves typical users doing typical tasks.

A **usability test** involves **typical users doing typical tasks.**

Researchers usually:

- observe users
- record errors
- measure time
- ask for feedback

Typical test conditions:

- controlled setting
- representative users
- representative tasks
- around **5–10 participants** in practice

Usability testing mainly measures **performance, satisfaction, and error rates.**

6. Important Evaluation Terms

Reliability whether the method gives the same results on different occasions.

Whether the method gives the **same results on different occasions.**

Validity whether the method measures what it is supposed to measure.

Whether the method measures **what it is supposed to measure**.

Ecological Validity

Whether the testing environment is **realistic** and does not distort results.

These are very common MCQ terms.

7. Nielsen's Important Heuristics

These are very important for exams.

Visibility of System Status

The system should keep users informed about what is happening through feedback.

Consistency and Standards

Similar actions and words should behave the same way across the interface.

Error Prevention

The design should reduce the chance of mistakes happening.

Recognition Rather Than Recall

Users should not have to remember too much information. Options should be visible.

Help Users Recover from Errors

Error messages should be clear and helpful.

8. Ethics in Evaluation

When testing with users, ethics matter.

Participants should:

- know why the study is happening
- know what they will do
- be allowed to stop at any time

- have privacy protected

This is done using an **informed consent form**.

A major ethical concern in HCI is **user privacy and data protection**.

9. Other Important Exam Points

A/B Testing

Used to compare **two design alternatives** and see which works better.

Eye-Tracking Heatmaps

Show **where users focus their attention on a screen**.

Progress Bar

A progress bar is an example of **visual feedback**.

Task Completion Rate

Shows how effectively users achieve their goals.

10. Benefits of Evaluation

- Finds problems early
 - Improves usability
 - Helps designers make better decisions
 - Increases user satisfaction
 - Reduces costly mistakes later
-

Quick Exam Summary

Key terms:

Evaluation
Usability Testing
Heuristic Evaluation

Field Study
Formative Evaluation
Summative Evaluation
Reliability
Validity
Ecological Validity
Error Prevention
Visibility of System Status
Informed Consent
A/B Testing

Possible Exam Questions

1. What is evaluation in HCI?
 2. What is usability testing?
 3. Difference between formative and summative evaluation
 4. What is heuristic evaluation?
 5. What is reliability?
 6. What is validity?
 7. What does visibility of system status mean?
 8. What is error prevention?
 9. What is informed consent?
 10. What is a field study?
-

Lesson 6: UX in the Software Life Cycle / Agile UX

1. What is UX in the Software Life Cycle?

UX should be included **throughout the software life cycle**, not added only at the end.

This means user experience work should happen during:

- planning
- design
- development
- testing
- maintenance

A requirement is a statement about what a product should do or how it should perform.

2. What is Agile Development?

Agile development is a way of **building** software in **short iterations**.

Main features:

- short development cycles
- repeated feedback
- changing priorities
- quick improvement

Examples of agile methods:

- Scrum
- XP
- Kanban
- DSDM

3. What is Agile UX?

Agile UX combines:

- **interaction design / UX**
with
- **agile software development**

Its purpose is to balance:

- good user research
- fast development
- frequent user feedback

Agile UX focuses on the **product**, not just documentation.

4. Key Ideas of Agile UX

Agile UX includes:

- user research
- rapid iterations
- evolving requirements

- cross-functional teamwork
- continuous feedback

Important idea: requirements are **not always fixed at the start**. They can change and be refined during development.

5. User Research in Agile UX

User research helps understand:

- users
- tasks
- context of use

Important point:

Detailed research is often too big for one short sprint.

So:

- some research may be done in **Iteration 0**
- more research continues during development

6. Iteration 0

Iteration 0 is early work done **before implementation starts**.

It may include:

- initial user research
- planning
- understanding requirements
- setting product vision

This is important because not all UX work fits inside short agile cycles.

7. Parallel Tracks Approach

This is one of the most important ideas in this lesson 🧐🧐

In **parallel tracks**:

- **designers** work slightly ahead
- **developers** build what **designers** prepared earlier

Example:

- Designers prepare designs for Cycle 2
- Developers build features from Cycle 1

This helps UX and development happen together without slowing everything down.

8. Advantages of Parallel Tracks

Parallel tracks help because:

- no design time is wasted on features not built
 - usability testing can happen alongside development
 - developers get feedback quickly
 - teams can respond to problems earlier
-

9. Documentation in Agile UX

Agile encourages **minimal documentation**.

The idea is:

- spend less time over-writing documents
- spend more time designing, discussing, and improving the product

Important question:

Only create documentation **when needed**.

10. Technical Debt in UX

UX debt happens when teams make quick design compromises that cause bigger usability problems later.

Causes:

- rushing design
- ignoring UX
- poor long-term planning
- repeated shortcuts

This is like technical debt, but for **user experience**.

11. Lean UX

Lean UX means building and releasing quickly to get **real user feedback**.

It follows:

- build
- measure
- learn

A **minimum viable product (MVP)** is often used to test assumptions early.

12. Discovering Requirements

Requirements help describe what the product must do.

Types of requirements:

Functional Requirements

What the system should do.

Example:

- browse products
- search furniture
- make payment

Data Requirements

What data the system must store.

Example:

- customer accounts

- product details
- order history

Context / Environment Requirements

Where and how the system will be used.

Examples:

- noisy place
 - mobile use
 - collaboration
 - privacy
 - training needs
 - technical compatibility
-

13. User Stories

A **user story** is a short requirement written from the user's view.

Format:

As a **[role]**
I want **[action]**
So that **[benefit]**

Example:

As a **customer**
I want to **filter furniture by category**
So that **I can find products faster.**

14. Personas and Scenarios

Personas

Fictional characters representing typical users.

Scenarios

Descriptions of how users carry out tasks.

Together, personas and scenarios help **bring requirements to life**.

15. Choosing Among Design Alternatives

Designers should not just keep the first idea.

They should:

- consider alternatives
- test with users or peers
- compare options

One method is **A/B testing**, which compares two alternatives to see which works better.

16. Risks if UX is Ignored

If UX is ignored during development:

- the product may be hard to use
- users may get confused
- errors may increase
- satisfaction may drop

Your lab also asks this directly: identify one risk if UX is ignored during implementation.

17. UX in Testing and Maintenance

Testing Phase

UX testing should check:

- task completion time
- error rate
- customer satisfaction

Maintenance Phase

After launch, UX feedback can still be gathered using:

- feedback forms

- analytics dashboards
- customer behaviour data

This feedback is used for future improvements.

18. Benefits of Integrating UX into Development

- better usability
- fewer design problems later
- faster improvement
- better teamwork between designers and developers
- better final product

Quick Exam Summary

Key terms:

Agile Development

Agile UX

Iteration 0

Parallel Tracks

Lean UX

UX Debt

Requirements

Functional Requirements

User Stories

Personas

Scenarios

A/B Testing

Possible Exam Questions

1. What is Agile UX?
2. What is Iteration 0?
3. What is the parallel tracks approach?
4. What is UX debt?
5. What are functional requirements?
6. What is a user story?
7. Why should UX be integrated into the software life cycle?
8. What happens if UX is ignored during implementation?

9. What is Lean UX?
 10. How can UX feedback be collected after launch?
-

Lesson 7 — Visualisation / Good UX through Visual Design

1. What is Visualisation?

Visualisation is the process of turning information into a **visual form** so users can understand it more easily.

It helps users:

- observe data quickly
- notice patterns
- make decisions
- understand complex information

Visualisation transforms information into a visual form so hidden features in data can be seen more clearly.

2. Why Visualisation is Important

Visualisation is important because it helps users:

- understand information faster
- compare items easily
- notice trends and differences
- reduce mental effort
- make better decisions

In HCI, good visualisation improves **usability and user experience**.

3. Information Visualisation (InfoVis)

Information Visualisation means using **interactive visual representations of abstract data** to help thinking and understanding.

It is used to help users find:

- trends
 - clusters
 - gaps
 - outliers
 - comparisons
-

4. What Good Visualisation Should Do

A good visual display should help users:

- see important information quickly
 - understand relationships
 - compare alternatives
 - avoid confusion
 - focus attention on what matters
-

5. Examples from the Lecture

Minard's Map

A famous visualisation showing **Napoleon's March**.

It is important because it shows a lot of information in one image:

- army size
- location
- direction
- temperature
- retreat path

Why it matters:

It proves that one good visual can communicate a lot of information clearly.

Tube Map The tube map is an example of simplifying reality to improve understanding.

The Tube Map is an example of simplifying reality to improve understanding.

Why it is useful:

- easier to read than a fully geographic map
- supports navigation
- focuses on what users need most

6. Visualisation in Interface Design

Visualisation can improve interfaces by making information easier to scan.

Examples:

- product categories shown with icons/images
- side-by-side comparisons
- coloured labels
- progress indicators
- visual feedback during checkout

Your lab specifically connects visualisation with a furniture store application and asks how visuals improve navigation, comparison, and decision support.

7. Text List vs Visual Grid

This is a very likely exam or coursework-style point.

Text-Based List

- simple
- compact
- useful for detailed information
- slower for visual scanning

Visual Grid

- uses icons or images
- easier to browse quickly
- better for product exploration
- more attractive and intuitive

For browsing things like sofas, tables, or beds, a **visual grid** usually gives better usability because users can recognize items faster.

8. Product Information Display

Raw product data should not just be dumped as plain text.

Instead, it can be shown visually using:

- icons
- tables
- colour coding
- labels
- comparison panels

Examples:

- stock status with colour
- dimensions in a neat table
- price highlighted clearly
- material shown with icons or badges

9. Decision Support Through Visualisation

Visualisation helps users choose between options.

Useful methods:

- comparison tables
- side-by-side product comparison
- filters
- charts
- highlighted differences

This is useful in shopping systems where users need to compare multiple items easily.

10. Visual Feedback in Interfaces

Visual feedback tells users what is happening.

Examples:

- progress bars

- confirmation ticks/icons
- highlighted errors
- loading indicators

These improve:

- trust
- confidence
- satisfaction
- clarity during tasks like checkout

11. Why Visual Cues Matter

Visual cues help users:

- know what to do next
- understand system status
- avoid mistakes
- feel more in control

This links with the heuristic **visibility of system status**, where the system keeps users informed through timely feedback.

12. Visualisation and UX

Good visualisation improves UX because it makes systems:

- clearer
- easier to navigate
- faster to understand
- more engaging
- less mentally tiring

Poor visualisation can confuse users and increase cognitive load.

13. Important Exam Ideas

Remember these:

- visualisation helps users **see patterns and relationships**

- good visuals support **decision-making**
- visual grids are often better than plain text lists for browsing products
- comparison tables help users compare alternatives
- progress bars and confirmation icons are forms of **visual feedback**
- colour coding can improve clarity, but must be used carefully

Quick Exam Summary

Key terms:

Visualisation

Information Visualisation (InfoVis)

Patterns

Comparison

Decision Support

Visual Grid

Comparison Table

Colour Coding

Visual Feedback

Progress Bar

Possible Exam Questions

1. What is visualisation?
2. What is Information Visualisation?
3. Why is visualisation important in HCI?
4. Difference between a text list and a visual grid
5. How can visualisation support decision-making?
6. Give examples of visual feedback
7. Why are progress bars important?

Lesson 8 — Computer Graphics: Line and Circle Drawing Algorithms

1. Why Line and Circle Drawing Matter

Lines and circles are **basic geometric shapes** used in computer graphics and HCI.

They are important for:

- interface design

- sketching tools
 - drawing programs
 - CAD software
 - digital whiteboards
 - touch and pen-based interaction
-

2. What is the Line Drawing Problem?

In computer graphics, a line is drawn on a **pixel grid**.

The problem is:

given only the two end points, how do we choose the intermediate pixels so the line looks as close as possible to the ideal line?

This is called **rasterization**.

3. DDA Algorithm

What is DDA?

DDA stands for **Digital Differential Analyzer**.

It is an **incremental line drawing algorithm** that calculates points along a line step by step.

It uses:

- slope
 - x and y increments
 - floating-point arithmetic
-

Main Idea of DDA

Starting from one end point:

- calculate $dx = x_2 - x_1$
- calculate $dy = y_2 - y_1$
- find number of steps
- calculate x increment and y increment
- keep adding increments until the final point is reached

Important DDA Facts

- used to draw **straight lines**
- mainly uses **floating-point arithmetic**
- simple and easy to understand
- works step by step from previous point to next point

Advantage of DDA

- simple to implement
- easy to understand
- can draw lines with different slopes

Disadvantage of DDA

Main disadvantage:

- it uses **floating-point operations**, which are slower

Also:

- rounding can cause **round-off errors**
- accuracy may reduce for some lines

4. Slope in DDA

The **slope (m)** helps decide how the line progresses.

Important exam point:

In DDA, slope determines **whether x or y changes more significantly at each step.**

Your quiz simplifies this as: slope determines **whether to increment x or y at each step.**

5. Bresenham's Line Algorithm

What is Bresenham's Algorithm?

Bresenham's Line Algorithm is another way to draw lines on a pixel grid.

It is more efficient because it uses:

- **integer arithmetic**
 - simple additions and subtractions
 - a decision variable to choose the next pixel
-

Why Bresenham is Important

Compared to DDA:

- faster
 - more efficient
 - avoids floating-point calculations
 - better suited to raster displays
-

Main Idea of Bresenham

At each step, the algorithm decides:

- should the next pixel go straight
- or diagonally

It uses a **decision parameter** to keep the line close to the ideal path.

6. DDA vs Bresenham

DDA

- uses floating-point arithmetic
- simpler
- slower

Bresenham

- uses integer arithmetic
- faster
- more efficient

Important exam point:

DDA is slower because it uses floating-point arithmetic.

7. Circle Drawing Algorithms

Circles are also drawn on a pixel grid, so algorithms are needed to choose the best pixels.

The main one in your notes is the:

Midpoint Circle Algorithm

Also called **Bresenham's Circle Algorithm**.

8. Midpoint Circle Algorithm

What it Does

The Midpoint Circle Algorithm draws a circle using:

- integer arithmetic
 - symmetry
 - a decision parameter
-

Main Idea

A circle is symmetric in **8 directions**.

So the algorithm:

- calculates points for one part of the circle
- then reflects them to the other parts

This makes circle drawing much faster.

Decision Parameter in Midpoint Circle

The decision parameter is used to decide:

- whether to move **horizontally**
- or **diagonally** when selecting the next pixel

This is a very common MCQ point.

Important Property Used

The Midpoint Circle Algorithm uses **symmetry**.

That is one of the most important facts to memorize.

9. Output of Midpoint Circle Algorithm

The algorithm produces:

- a set of points forming a circle
 - using symmetry
 - on a raster display
-

10. Anti-Aliasing

What is Aliasing?

Aliasing is the **jagged or stair-step appearance** seen when diagonal lines or curves are drawn on a pixel grid.

This makes shapes look rough instead of smooth.

What is Anti-Aliasing?

Anti-aliasing is a technique used to make lines and curves look smoother.

It works by:

- blending edges
- using intermediate colours or transparency
- reducing the jagged appearance

Why Anti-Aliasing is Important

It improves:

- visual clarity
- readability
- appearance of graphics
- overall user experience

11. HCI Connection

These algorithms matter in HCI because they support:

- accurate drawing tools
- sketch interfaces
- shape rendering
- smoother visual displays
- better interaction in design software and digital interfaces

12. Must-Remember Exam Points

Memorize these:

- **DDA = Digital Differential Analyzer**
- DDA is used to **draw straight lines**
- DDA mainly uses **floating-point arithmetic**
- DDA's disadvantage = floating-point operations are slower
- Bresenham's line algorithm is more efficient for integer-based line drawing
- Midpoint Circle Algorithm uses **symmetry**
- Midpoint Circle Algorithm uses a **decision parameter** to choose the next pixel
- Anti-aliasing reduces jagged edges

Quick Exam Summary

Key terms:

DDA
Digital Differential Analyzer

Bresenham's Line Algorithm
Midpoint Circle Algorithm
Decision Parameter
Symmetry
Rasterization
Aliasing
Anti-Aliasing

Possible Exam Questions

1. What does DDA stand for?
 2. What is the purpose of the DDA algorithm?
 3. What is the main disadvantage of DDA?
 4. Which algorithm is more efficient for integer line drawing?
 5. What does the decision parameter do in the Midpoint Circle Algorithm?
 6. Which property of circles is used in the Midpoint Circle Algorithm?
 7. What is aliasing?
 8. What is anti-aliasing?
 9. Difference between DDA and Bresenham's algorithm
 10. Why are line and circle algorithms important in HCI?
-

Lesson 9 — Digital Image Processing

1. What is a Digital Image?

A **digital image** is a **2-dimensional matrix of intensity values**. These values may represent:

- grayscale
- colour
- opacity

Image coordinates are integers, and each small unit is called a **pixel**.

2. What is Digital Image Processing?

Digital Image Processing means using algorithms to **change or improve an image**.

Important idea:

- **input = image**
- **output = image**

It is used for:

- image enhancement
 - editing
 - display
 - printing
 - compression
-

3. Why Image Processing is Important

Image processing is important because it helps:

- improve image quality
 - make details easier to see
 - reduce file size
 - support recognition and analysis
 - improve user experience in image-based systems
-

4. Common Types of Digital Images

Grayscale

- 1 value per pixel

RGB

- 3 values per pixel:
 - Red
 - Green
 - Blue

RGBA

- RGB plus **Alpha** (opacity/transparency)
-

5. Image Enhancement

Image enhancement improves the appearance of an image so users can see details more clearly.

Examples:

- contrast adjustment
- noise removal
- sharpening

Your materials show enhancement examples like **noise removal** and **contrast adjustment**.

Noise Removal

Noise means unwanted specks or distortion in an image.

Purpose:

- make the image clearer
 - remove unnecessary visual disturbance
 - improve visibility of details
-

Contrast Adjustment

This improves the difference between light and dark areas.

Purpose:

- make details easier to notice
 - improve visibility
 - help users understand the image better
-

6. Image Compression

Compression reduces the file size of an image.

Why it matters:

- faster loading

- less storage use
- better web performance

Your furniture-store lab directly links compression to balancing **image quality and loading speed**.

JPEG vs PNG

JPEG

- smaller file size
- good for photos
- uses lossy compression

PNG

- higher quality
 - better for graphics and transparency
 - larger file size
-

Risk of Too Much Compression

If compression is too aggressive:

- image quality drops
 - details become unclear
 - users may lose trust in the product image
-

7. Segmentation

Segmentation means dividing an image into meaningful parts.

Example:

- separating a furniture item from the background

Why useful:

- cleaner product display
- easier analysis

- supports automatic processing
-

8. Object Recognition

Object recognition means identifying what appears in an image.

Example:

- recognising whether the object is a sofa, chair, or table

Why useful:

- automatic tagging
 - product categorisation
 - smarter search features
-

9. Edge Detection

Edge detection finds boundaries in an image.

Purpose:

- highlight shape outlines
- identify object borders
- support recognition and analysis

This is often used in medical and image analysis systems too.

10. Region Detection / Segmentation

This means identifying a whole area or object inside an image.

Purpose:

- divide image into meaningful regions
 - isolate important parts
 - support analysis and interaction
-

11. Colour Image Processing

Colour processing helps represent items accurately.

RGB

Used for digital screens and online images.

CMYK

Used more in printing.

Why Colour Fidelity is Important

Colour fidelity means colours should appear as accurately as possible.

This is important because:

- customers expect the real product colour
 - wrong colours reduce trust
 - bad colour display may lead to disappointment or returns
-

12. Interactive Features Using Image Processing

Image processing supports interactive features like:

- zoom
- rotate
- AR preview
- detailed image viewing

These features improve the shopping experience by helping users inspect products more closely.

13. Applications of Image Processing

Your lecture gives several application areas:

- medicine
- GIS / weather / terrain

- law enforcement
 - HCI
 - face recognition
 - gesture recognition
-

14. HCI Connection

Digital image processing matters in HCI because it improves:

- how users view images
- how systems recognise visual input
- how clearly information is shown
- how interactive image features work

Examples in HCI:

- face recognition
 - gesture recognition
-

15. Must-Remember Exam Points

Memorize these:

- a digital image is a **2D matrix of intensity values**
 - image processing means **input image → processed image**
 - common operations:
 - noise removal
 - contrast adjustment
 - edge detection
 - segmentation
 - compression
 - segmentation separates meaningful parts of an image
 - object recognition identifies objects in an image
 - RGB is for screens; CMYK is for print
 - compression improves loading speed but too much reduces quality
-

Quick Exam Summary

Key terms:

Digital Image
Pixel
Image Processing
Image Enhancement
Noise Removal
Contrast Adjustment
Compression
JPEG
PNG
Segmentation
Object Recognition
Edge Detection
RGB
CMYK
Colour Fidelity

Possible Exam Questions

1. What is a digital image?
 2. What is digital image processing?
 3. What is the purpose of image enhancement?
 4. What is noise removal?
 5. What is image compression?
 6. Difference between JPEG and PNG
 7. What is segmentation?
 8. What is object recognition?
 9. Why is colour fidelity important?
 10. How does image processing improve user experience?
-

Lesson 10 — Java Basics for HCI / OOP in Java

1. What is Java?

Java is an **object-oriented programming language** used to build software applications.

It is important in your module because Java is used for:

- UI development
- Swing applications
- graphics programming
- interactive systems

2. What is the JVM?

JVM stands for **Java Virtual Machine**.

Purpose:

- runs Java bytecode
- provides the runtime environment for Java programs

A Java program is treated as a collection of **objects communicating with each other**.

3. Java is Object-Oriented

Java supports key OOP concepts:

- classes
- objects
- methods
- encapsulation
- inheritance
- polymorphism
- abstraction

These are very important for exams.

4. Class and Object

Class

A **class** is a **blueprint or template** for creating objects.

Example:

A Dog class may contain:

- breed
 - age
 - colour
 - barking()
-

Object

An **object** is an **instance of a class**.

Example:

If Dog is a class, then an individual dog created from it is an object.

5. Methods and Variables

Method

A **method** is a behaviour or action inside a class.

Example:

- barking()
 - hungry()
 - sleeping()
-

Types of Variables in Java

Local Variable

Declared inside a method.

Instance Variable

Declared inside a class but outside methods.

Class Variable

Declared with the `static` keyword.

6. Java Syntax Basics

Important syntax rules:

- Java is **case sensitive**
- class names usually start with **Upper Case**

- method names usually start with **lower case**
 - file name must match class name
 - execution starts from main()
-

7. The main() Method

Every Java program starts execution from:

```
public static void main(String[] args)
```

This is the entry point of the program.

8. Constructor

A **constructor** is a special method used when an object is created.

Important facts:

- constructor name must match the class name
 - it is called automatically when an object is created
 - a class can have more than one constructor
-

Example Idea

If class name is Book, then constructor is also:

```
Book()
```

Used to initialize the object.

9. Creating an Object

There are 3 steps:

1. Declaration

Create a variable of object type.

2. Instantiation

Use `new` keyword.

3. Initialization

Constructor is called.

Example:

```
Book book = new Book();
```

10. Encapsulation

Encapsulation means hiding sensitive data inside a class and controlling access to it.

Done by:

- making variables private
- using public **get** and **set** methods

Why important:

- protects data
 - controls access
 - improves code safety
-

11. Inheritance

Inheritance allows one class to inherit attributes and methods from another class.

Terms:

- **Superclass** = parent class
- **Subclass** = child class

Keyword:

`extends`

Why use inheritance?

- code reuse
 - avoids repetition
 - makes programs easier to organize
-

12. Polymorphism

Polymorphism means **many forms**.

It happens when related classes use the **same method name** but behave differently.

Example:

Different animals may all have:

```
animalSound()
```

But each gives a different output.

Why important:

- one action can work in different ways
-

13. Abstraction

Abstraction means hiding unnecessary details and showing only essential information.

It can be done using:

- abstract classes
- abstract methods

Important points:

- abstract class cannot be instantiated directly
 - abstract method has no body
 - subclass must provide the method body
-

14. Access Modifiers

Java uses access modifiers to control visibility.

Main ones:

- public
- private
- protected
- default (package-level)

Very common exam point:

- private = visible only inside the class
 - public = visible everywhere
-

15. Non-Access Modifiers

Examples:

- static
- final
- abstract

Important meanings:

- static = belongs to class, not object
 - final = cannot be changed / extended in some contexts
 - abstract = incomplete and meant to be extended
-

16. Scanner Class

The **Scanner** class is used to take **user input**.

Package:

java.util

Example use:

```
Scanner myObj = new Scanner(System.in);
```

Used to read:

- integers
- strings

- other input types
-

17. ArrayList

ArrayList is a resizable array in Java.

Difference from normal array:

- normal array size is fixed
- ArrayList size can grow or shrink

Package:

java.util.ArrayList

18. Import Statement

Used to include classes/packages.

Example:

```
import java.util.Scanner;  
import java.util.ArrayList;
```

This is needed before using those classes.

19. Why Java Matters in HCI

Java is useful in HCI because it supports:

- GUI development
- event-driven programming
- graphics
- interactive software

It is the foundation for later topics like:

- Java Swing
- graphics drawing
- UI implementation

20. Must-Remember Exam Points

Memorize these:

- **JVM** = Java Virtual Machine
- Java is an **object-oriented language**
- **class** = blueprint
- **object** = instance of class
- `main()` is where execution starts
- **constructor** has same name as class
- **encapsulation** = hide data using private + getter/setter
- **inheritance** uses extends
- **polymorphism** = same method, different behaviour
- **abstraction** = hide details, show essentials
- **Scanner** = user input
- **ArrayList** = resizable array

Quick Exam Summary

Key terms:

Java
JVM
Class
Object
Method
Constructor
Encapsulation
Inheritance
Polymorphism
Abstraction
Access Modifiers
Scanner
ArrayList

Possible Exam Questions

1. What does JVM stand for?
2. What is a class in Java?
3. What is an object?

4. What is a constructor?
 5. What is encapsulation?
 6. What is inheritance?
 7. What is polymorphism?
 8. What is abstraction?
 9. What is the purpose of Scanner?
 10. Difference between array and ArrayList
-

Lesson 11 — Java Swing & Building GUIs

1. What is Java Swing?

Java Swing is a Java library used to build **window-based graphical user interfaces (GUIs)**.

It is part of **Java Foundation Classes (JFC)** and provides ready-made GUI components like:

- buttons
 - text fields
 - labels
 - menus
 - dialog boxes
-

2. Why Swing is Important in HCI

Swing is important because it helps developers create:

- interactive windows
- user input forms
- buttons and controls
- event-driven applications
- simple desktop interfaces

It is used to turn HCI designs into real software interfaces.

3. Common Swing Components

JFrame

A **JFrame** is the main window of a Swing application.

It is used to:

- hold components
 - display the GUI
 - act as the main container
-

JButton

A **JButton** is a clickable button.

Used for actions like:

- submit
- login
- calculate
- open dialog

Important method:

- `addActionListener()` → used to handle button clicks
-

JLabel

A **JLabel** displays text or images on the interface.

Example:

- “Username”
 - “Password”
 - “Welcome”
-

TextField

A **TextField** is used for single-line text input.

Example:

- username input
- search box

JTextArea

A **JTextArea** is used for multi-line text input.

JPasswordField

A **JPasswordField** is used for password input.

4. Common Component Methods

Important methods from Swing components:

- `add(Component c)` → adds a component
- `setSize(width, height)` → sets size
- `setLayout(...)` → sets layout manager
- `setVisible(true)` → makes component visible

These are very common in GUI setup.

5. Event Handling

Swing GUIs are **event-driven**.

That means the program reacts to user actions like:

- clicking a button
- typing in a field
- selecting an option

The most common event in your notes is:

ActionEvent

Handled using:

ActionListener

Example:

When a button is clicked, the program performs some action.

6. JOptionPane

JOptionPane is used to show standard dialog boxes.

Common uses:

- message dialog
- confirm dialog
- input dialog

Examples:

- error messages
- confirmation popups
- asking the user for input

Important methods:

- showMessageDialog(...)
 - showConfirmDialog(...)
 - showInputDialog(...)
-

7. Other Useful Swing Components

Your notes also include:

JScrollBar

Used for scrolling.

JToolBar

Used to place tools/buttons in a toolbar.

JDialog

Used for pop-up windows/dialog boxes.

JEditorPane

Used to display/edit content like text or HTML.

8. Graphics in Swing

Swing can also draw graphics using the **Graphics** class.

Important drawing methods:

- `drawString()`
- `drawRect()`
- `fillRect()`
- `drawOval()`
- `fillOval()`
- `drawLine()`
- `drawArc()`
- `fillArc()`

These are used in graphical and interactive programs.

9. DDA and Midpoint Circle in Java

Your Java Swing practicals also connect GUI programming with graphics algorithms.

Examples included in your notes:

- **DDA line drawing**
- **Midpoint circle drawing**

This means Swing is not just for forms — it is also used for:

- drawing shapes
 - implementing graphics algorithms
 - visualizing output
-

10. Transformations in Java Swing

Using **Graphics2D** and **AffineTransform**, Swing can perform:

Translation

Moves an object from one position to another.

Scaling

Changes the size of an object.

Rotation

Turns an object around an angle.

These are important in graphics and visual applications.

11. Typical Structure of a Swing Program

A simple Swing app often does this:

1. Create a JFrame
 2. Create components (JButton, JTextField, etc.)
 3. Add components to the frame
 4. Set size and layout
 5. Make frame visible
 6. Add event handling if needed
-

12. Why Swing Matters for the Exam

Swing connects directly to:

- GUI design
- event-driven programming
- user interaction
- graphics
- practical interface building

So it links Java programming with HCI concepts.

13. Must-Remember Exam Points

Memorize these:

- **Java Swing** is used to create **window-based applications**
- **JFrame** = main window
- **JButton** = clickable button

- **JTextField** = single-line input
 - **JLabel** = displays text/image
 - **JPasswordField** = password input
 - **JOptionPane** = dialog boxes/messages
 - `addActionListener()` handles button actions
 - `setVisible(true)` shows the GUI
 - Graphics methods are used to draw shapes/text
 - Graphics2D + AffineTransform support translation, scaling, and rotation
-

Quick Exam Summary

Key terms:

Java Swing
JFrame
JButton
JLabel
JTextField
JPasswordField
JOptionPane
ActionListener
Graphics
Graphics2D
Translation
Scaling
Rotation

Possible Exam Questions

1. What is Java Swing?
 2. What is the purpose of JFrame?
 3. What is JButton used for?
 4. What is JTextField used for?
 5. What is JOptionPane?
 6. What does `addActionListener()` do?
 7. What is event-driven programming?
 8. Name some Graphics methods in Java Swing.
 9. What is the purpose of Graphics2D?
 10. What transformations can be done in Java Swing?
-

Lesson 12 — OpenGL / JOGL Basics in Java

1. What is JOGL?

JOGL stands for **Java binding for OpenGL**. It is a Java library that allows Java programs to use the **OpenGL graphics API**.

It is used to create:

- 2D graphics
 - 3D graphics
 - visual and interactive applications
-

2. Why JOGL is Important

JOGL is important because it allows Java developers to:

- draw shapes using OpenGL
- build graphics applications
- create animated visual content
- apply transformations like scaling and rotation

It connects Java programming with **real-time graphics**.

3. OpenGL Setup in Java

To use JOGL, the required .jar libraries must be added to the Java project.

Examples from your notes:

- gluegen-rt.jar
- jogl-all.jar
- native JOGL libraries for the operating system

Important point:

The correct libraries must be added, and duplicate versions can cause problems.

4. GLEventListener

A JOGL class usually implements **GLEventListener**.

This interface provides key methods:

- `display()`
 - `init()`
 - `dispose()`
 - `reshape()`
-

5. Purpose of the GLEventListener Methods

display()

Used to draw graphics on the screen.
This is the most important drawing method.

init()

Used for initialization when OpenGL starts.

dispose()

Used for cleanup when the program closes.

reshape()

Used when the window changes shape or size.

6. GL2 Object

Inside `display()`, the program usually creates a **GL2** object.

This object is used to call OpenGL drawing commands.

Example idea:

```
final GL2 gl = drawable.getGL().getGL2();
```

7. Drawing Shapes in JOGL

JOGL can draw shapes using commands like:

- glBegin(...)
- glVertex...(...)
- glEnd()

These are used to draw:

- lines
 - triangles
 - polygons
 - squares
-

8. Drawing a Square with Lines

Your notes show a square being drawn by drawing **four separate lines**.

This shows how OpenGL uses vertices to define shapes.

Important idea:

A shape is built from points and edges.

9. GLProfile, GLCapabilities, and GLCanvas

These are important JOGL setup classes.

GLProfile

Used to select the OpenGL version/profile.

GLCapabilities

Stores graphics capability settings.

GLCanvas

A canvas where OpenGL graphics are drawn.

10. GLJPanel

Your notes mention that in newer JDK versions, **GLCanvas** may not always be supported, so **GLJPanel** can be used instead.

This is a useful exam detail.

11. Displaying JOGL in a Window

To display graphics:

1. create a GLProfile
 2. create GLCapabilities
 3. create a GLCanvas or GLJPanel
 4. create the drawing class
 5. add the event listener
 6. add canvas/panel to a JFrame
 7. make frame visible
-

12. Scaling in JOGL

Scaling changes the size of an object.

In JOGL, this can be done using:

```
gl.glScalef(...)
```

This makes objects:

- smaller
 - larger
 - stretched in different directions
-

13. Rotation in JOGL

Rotation turns an object around an axis.

In JOGL, this is done using:

`gl.glRotatef(...)`

This is used for:

- rotating triangles
- animations
- changing object orientation

14. Lighting in JOGL

Your notes also include **lighting** examples in JOGL.

Lighting is used to make graphics appear more realistic by affecting:

- brightness
- colour appearance
- visual depth

This is especially important in 3D scenes.

15. Animation in JOGL

Animation can be created by repeatedly updating the scene.

Your notes show this using:

- rotation angle updates
- FPSAnimator for continuous rendering

Important idea:

Animation happens by redrawing graphics many times with small changes.

16. Why OpenGL Matters in HCI

OpenGL / JOGL matters in HCI because it supports:

- visual interfaces
- real-time graphics
- interactive 2D and 3D systems

- animation
- graphical feedback

It is useful in applications like:

- games
- design tools
- simulations
- visual interfaces

17. Difference Between Swing Graphics and JOGL

Swing Graphics

- simpler
- used for basic 2D GUI drawing
- uses Graphics / Graphics2D

JOGL / OpenGL

- more powerful
- supports advanced 2D and 3D graphics
- used for real-time rendering

This comparison is useful for understanding when each is used.

18. Must-Remember Exam Points

Memorize these:

- **JOGL** = Java binding for OpenGL
- JOGL is used for **2D and 3D graphics**
- GLEventListener includes:
 - display()
 - init()
 - dispose()
 - reshape()
- display() is the main drawing method
- GLCanvas is used to draw OpenGL graphics
- GLJPanel can be used instead of GLCanvas in newer JDK situations
- glScalef() = scaling
- glRotatef() = rotation

- FPSAnimator supports animation
-

Quick Exam Summary

Key terms:

JOGL
OpenGL
GLEventListener
display()
init()
dispose()
reshape()
GL2
GLProfile
GLCapabilities
GLCanvas
GLJPanel
Scaling
Rotation
Lighting
Animation

Possible Exam Questions

1. What is JOGL?
2. What is JOGL used for?
3. What is the purpose of GLEventListener?
4. Name the main methods of GLEventListener
5. What does display() do?
6. What is GLCanvas?
7. What is the purpose of GLProfile?
8. What does glScalef() do?
9. What does glRotatef() do?
10. Why is JOGL useful in graphics applications?